

Módulo 9: Evaluación

Text Mining en Python

Antonio Ramón Vázquez Ramírez

2026-09-08

- 1 Carga de datos
 - 1.1 Descripción de los datasets
- 2 Pregunta 1. Text Mining
 - 2.1 Descripción de las columnas
 - 2.1.1 Términos más frecuentes.
 - 2.1.2 Buscar palabras clave:
 - 2.1.3 Visualizar las coocurrencias.
 - 2.1.4 Reconocimiento de entidades nombradas (NER).
 - 2.1.5 Análisis de Casos Específicos
 - 2.1.6 Word embeddings
- 3 Pregunta 2. Clasificación de Texto: Análisis de sentimiento
 - 3.1 Descripción de las columnas
 - 3.2 Se pide
- 4 Pregunta 3. Grandes Modelos de Lenguaje (LLM)
 - 4.1 Descripción de las columnas
 - 4.2 Se pide
 - 4.2.1 Clustering con BERT
 - 4.2.2 Clustering con Nomic Bert
 - 4.2.3 Clasificación de texto con Qwen3
 - 4.2.4 Comparar la clasificación de Qwen3 con el clustering de BERT
 - 4.2.5 Comparar la clasificación de Qwen3 con el clustering de Nomic Bert
- 5 Pregunta 4. Grandes Modelos de Lenguaje (LLM)
 - 5.1 Páginas web (URL) a procesar
 - 5.2 Se pide
 - 5.2.1 Gemini
 - 5.2.2 Ollama

Carga de entorno python y librerías

1 Carga de datos

Desde la sesión de R, cargamos el archivo `evaluacion.RData`, que contiene tres datasets: `prado`, `reviews` y `documentos`. Exportamos dichos datasets como CSV.

Desde la sesión de R

Hide

```
setwd("G:/Mi unidad/Antonio/estudios/BigData&DataScience/M9/evaluacion")
```

Hide

```
load("evaluacion.RData")

readr::write_delim(prado, "prado.csv", delim = "\t")
readr::write_delim(reviews, "reviews.csv", delim = "\t")
readr::write_delim(documentos, "documentos.csv", delim = "\t")
```

Cargamos los datos.

Hide

```
from pandas import read_csv

prado = read_csv("prado.csv", sep = "\t" , dtype = {"status":str})
reviews = read_csv("reviews.csv", sep = "\t")
documentos = read_csv("documentos.csv", sep = "\t")
```

Comprobamos que los datos se han cargado

[Hide](#)

```
print(f"Tuits de Prado: {len(prado)}")
```

```
## Tuits de Prado: 1038
```

[Hide](#)

```
print(f"Reviews: {len(reviews)}")
```

```
## Reviews: 13000
```

[Hide](#)

```
print(f"Documentos: {len(documentos)}")
```

```
## Documentos: 120
```

1.1 Descripción de los datasets

- **prado:** Cuenta de X (antiguo Twitter) del Museo del Prado: 1.038 tuits (desde 01-04-2025 al 31-07-2026).
- **reviews:** Contiene 13.000 críticas sobre el alojamiento en diversas ciudades de Andalucía.
- **documentos:** Contiene 120 oraciones.

2 Pregunta 1. Text Mining

El dataset `prado` contiene 1.038 tuits, de la cuenta del Museo del Prado.

2.1 Descripción de las columnas

- **status:** Identificador del tuit
- **fecha:** Fecha de publicación del tuit. Desde el 01-04-2025 hasta el 31-07-2026.
- **texto:** El texto del tuit.

Para realizar text mining al texto de los tuits, es importante depurar previamente el texto.

Como, por ejemplo, eliminar del texto: usuarios de X, hashtags, URLs, ...

Para realizar la limpieza del texto, podemos utilizar el siguiente script:

[Hide](#)

```
# === Limpieza de Texto ===

import re, string

prado['textmining'] = prado['texto']

# Quitar "@usuario"
prado['textmining'] = prado['textmining'].str.replace(
    r'@\w+', '', regex=True
)

# Quitar "#hashtag"
prado['textmining'] = prado['textmining'].str.replace(
    r'#\w+', '', regex=True
)

# Quitar "URL"
prado['textmining'] = prado['textmining'].str.replace(
    r'http\S*', '', regex=True
)

# Quitar "e-Mail"
prado['textmining'] = prado['textmining'].str.replace(
    r'\w+@\w+\.\w+', '', regex=True
)

# Si fuera necesario, añadir espacio después de "punto", "coma", "punto y coma", "dos puntos"
prado['textmining'] = prado['textmining'].str.replace(
    r'([.,;:])([a-zA-ZáéíóúñÁÉÍÓÚÑ])', r'\1 \2', regex=True
)

# Conservar letras, dígitos, espacios en blanco, saltos de línea y caracteres de puntuación
prado['textmining'] = prado['textmining'].str.replace(
    r'^\wÃ-ÿ' + re.escape(string.punctuation) + r'\s', '', regex=True
)

# Quitar símbolos
prado['textmining'] = prado['textmining'].str.replace('í', '')

# Quitar espacios en blanco al inicio y final del texto
prado['textmining'] = prado['textmining'].str.strip()
```

Realizar text mining a los tuits de la cuenta prado , desde 01-06-2025 hasta 31-07-2026.

Empezamos realizando el filtrado temporal que requiere

[Hide](#)

```
import pandas as pd
prado["fecha"] = pd.to_datetime(prado["fecha"])
prado_filtrado = prado[(prado["fecha"] >= "2025-06-01") & (prado["fecha"] <= "2026-07-31")].copy()

print(f"Dataset listo. Tuits totales en el rango: {len(prado_filtrado)}")
```

```
## Dataset listo. Tuits totales en el rango: 864
```

Utilizar las librerías spacy y nltk , según proceda.

Trabajar con la columna "textmining" . Operar con sustantivos, adjetivos y nombres propios.

Se pide los siguientes apartados expuestos a continuación, cada uno realizado como una sección independiente

2.1.1 Términos más frecuentes.

Extraemos únicamente los tokens y lemas correspondientes a Sustantivos, Adjetivos y Nombres Propios (NOUN , ADJ , PROPN) y eliminamos las stop words.

Los tokens se utilizaran para el calculo de colocaciones, mientras que los lemas se utilizaran para las frecuencias y concurrencias

Hide

```
import spacy
from collections import Counter
import matplotlib.pyplot as plt

# Cargar el modelo en español
nlp = spacy.load("es_core_news_sm")

tokens = []
lemas = []

# Procesar con pipe para optimizar recursos
for doc in nlp.pipe(prado_filtrado["textmining"], disable=["ner"]):
    for token in doc:
        if token.pos_ in ["NOUN", "ADJ", "PROPN"] and not token.is_stop and not token.is_punct:
            tokens.append(token.text)
            lemas.append(token.lemma_.lower())

# Obtener frecuencias
freq_lemas = Counter(lemas)
df_lemas = pd.DataFrame(freq_lemas.most_common(25), columns=["Lema", "Frecuencia"])

print("Top 10 Lemas:")
```

Top 10 Lemas:

Hide

```
print(df_lemas.head(10))
```

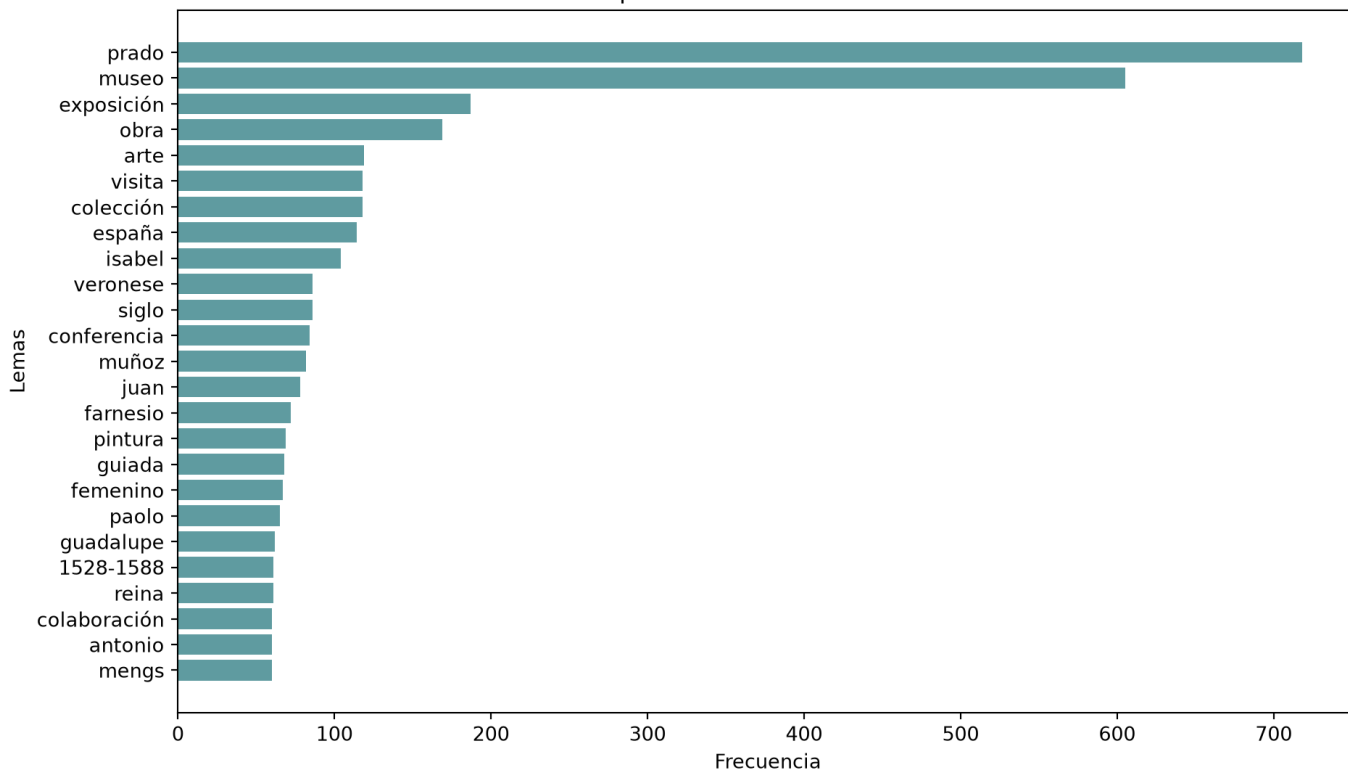
```
##      Lema  Frecuencia
## 0      prado         718
## 1      museo         605
## 2  exposición         187
## 3       obra         169
## 4       arte         119
## 5      visita         118
## 6  colección         118
## 7    españa         114
## 8     isabel         104
## 9   veronese          86
```

Tambien podemos verlos en una representación gráfica

Hide

```
plt.figure(figsize=(10, 6))
plt.barh(df_lemas["Lema"], df_lemas["Frecuencia"], color="cadetblue")
plt.gca().invert_yaxis()
plt.xlabel("Frecuencia")
plt.ylabel("Lemas")
plt.title("Top 25 Lemas más frecuentes")
plt.tight_layout()
plt.show()
```

Top 25 Lemas más frecuentes



prado (718 apariciones) y museo (605 apariciones): Son, con diferencia, los términos más frecuentes. Lo cual es coherente con el dataset que estamos analizando. Por otro lado, el análisis lingüístico permite desvelar el foco del periodo analizado (2025-2026), destacando los nombres propios 'Isabel' y 'Veronese', dando a entender las principales líneas expositivas y de comunicación del museo en esas fechas.

Como detalle, podemos notar el rango temporal (1528-1588), siendo este el periodo de vida del autor Paolo Veronese.

Hide

```
from spacy.matcher import Matcher
# 1. Configurar Matcher para detectar palabras en mayúsculas (Siglas)
matcher_siglas = Matcher(nlp.vocab)
pattern_siglas = [{"IS_UPPER": True}]
matcher_siglas.add("patron_siglas", [pattern_siglas])

siglas_encontradas = []
for doc in nlp.pipe(prado_filtrado["textmining"], disable=["ner"]):
    matches = matcher_siglas(doc)
    for match_id, start, end in matches:
        # Extraemos el texto de la coincidencia (en minúsculas o mayúsculas originales)
        # Usamos .text para quedarnos con el string
        siglas_encontradas.append(doc[start:end].text)

# 2. Obtener frecuencias de las siglas encontradas
freq_siglas = Counter(siglas_encontradas)

# 3. Generar DataFrame con las 15 siglas más comunes
df_siglas = pd.DataFrame(freq_siglas.most_common(15), columns=["Sigla", "Frecuencia"])

# 4. Imprimir el resultado de forma elegante en consola
print("Top 15 Siglas más encontradas en el corpus:")
```

```
## Top 15 Siglas más encontradas en el corpus:
```

Hide

```
print(df_siglas.to_string(index=False))
```

##	Sigla	Frecuencia
##	A	66
##	XXI	31
##	XIX	24
##	III	24
##	XVIII	19
##	IV	15
##	XVII	8
##	VII	8
##	XIV	7
##	XX	6
##	XV	5
##	V	5
##	N1	5
##	II	4
##	XVI	4

Podemos ver que la mayoría de siglas representan numeros romanos. Esto es lo habitual cuando se mencionan siglos o nombres de reyes sucesivos. Contamos con una anomalia, y es el hecho de que la sigla mas encontrada corresponde a "A". Vamos a analizar a que puede deberse con algunos ejemplos

Hide

```
ejemplos_A = []

for doc in nlp.pipe(prado_filtrado["textmining"], disable=["ner"]):
    for token in doc:
        # Si el token es exactamente "A" en mayúscula
        if token.text == "A" and token.is_upper:
            # Extraer un trozo de texto alrededor (contexto de 3 palabras antes y después)
            start = max(0, token.i - 3)
            end = min(len(doc), token.i + 4)
            contexto = " ".join([t.text for t in doc[start:end]])
            ejemplos_A.append((doc.text, contexto))

# 2. Imprimir Los 10 primeros resultados
print(f"Total de coincidencias de 'A' como sigla: {len(ejemplos_A)}\n")
```

```
## Total de coincidencias de 'A' como sigla: 66
```

Hide

```
for idx, (tuit, ctx) in enumerate(ejemplos_A[:10]):
    print(f"Ejemplo {idx+1}:")
    print(f" > Contexto: ... {ctx} ...")
    print(f" > Tuit original: {tuit[:120]}...")
    print("-" * 60)
```

```

## Ejemplo 1:
## > Contexto: ... A solas con " ...
## > Tuit original: A solas con "Las meninas" de Velázquez (1656)...
## -----
## Ejemplo 2:
## > Contexto: ... Javier Fonseca
##
## A partir del 14 ...
## > Tuit original: Taller de escritura epistolar en el marco del programa "Prado Extendido", con Chiki Fabregat
y Javier Fonseca
##
## A partir ...
## -----
## Ejemplo 3:
## > Contexto: ... Astrit Schmidt-Burkhardt . A partir del 6 ...
## > Tuit original: XIV Cátedra del Prado: "Tiempo, la fuerza invisible. Conceptos imaginales en el curso de la
historia", dirigida por la h...
## -----
## Ejemplo 4:
## > Contexto: ... Arte " . A partir del 18 ...
## > Tuit original: Las esculturas de Juan Muñoz ya están en la puerta del Museo del Prado para la exposición "J
uan Muñoz. Historias de Arte...
## -----
## Ejemplo 5:
## > Contexto: ... Arte " . A partir del 18 ...
## > Tuit original: Exposición "Juan Muñoz. Historias de Arte". A partir del 18 de noviembre en el Museo del Pra
do
##
## Con la colaboración de...
## -----
## Ejemplo 6:
## > Contexto: ... A partir de mañana ...
## > Tuit original: A partir de mañana se podrá visitar la exposición "Antonio Raphael Mengs (1728-1779)", que c
uenta con el patrocinio excl...
## -----
## Ejemplo 7:
## > Contexto: ... " A solas con Juan ...
## > Tuit original: "A solas con Juan Muñoz". Taller conducido por Iara Solano
##
## Del 19 al 22 de febrero. Plazas disponibles:...
## -----
## Ejemplo 8:
## > Contexto: ... el espacio . A propósito de Juan ...
## > Tuit original: Agenda del 2 al 8 febrero en el Museo del Prado
##
## Exposiciones
## - El Prado multiplicado. La fotografía como memoria compar...
## -----
## Ejemplo 9:
## > Contexto: ... para jóvenes " A solas con Juan ...
## > Tuit original: Taller de creación para jóvenes "A solas con Juan Muñoz", conducido por la investigadora Iar
a Solano
##
## Del 19 al 22 de fe...
## -----
## Ejemplo 10:
## > Contexto: ... para jóvenes " A solas con Juan ...
## > Tuit original: Taller de creación para jóvenes "A solas con Juan Muñoz", conducido por la investigadora Iar
a Solano
##
## Del 19 al 22 de fe...
## -----

```

Podemos observar, que es común que una oración empiece con la preposición “A”, por lo que esto explicaría como ha sido clasificado como el termino más frecuente entre las siglas

2.1.2 Buscar palabras clave:

- N-gramas: obtener bigramas, trigramas, ...
- Colocaciones: obtener bigramas, trigramas, ...

Ahora, analizamos los bigramas mas frecuentes.

[Hide](#)

```
from spacy.matcher import Matcher

def terminos_top(lista):
    term_freq = Counter(lista)
    df = pd.DataFrame.from_dict(term_freq, orient="index").reset_index()
    df.columns = ["termino", "freq"]
    return df.sort_values(by="freq", ascending=False).reset_index(drop=True)

# 1. Pasada "sucio" (sin stopwords) tal como hace el profesor
matcher_sucio = Matcher(nlp.vocab)

# Patrón base sin restricciones de Lema
pattern_sucio = [
    {"POS": {"IN": ["ADJ", "NOUN", "PROPN"]}},
    {"POS": {"IN": ["ADJ", "NOUN", "PROPN"]}}
]
matcher_sucio.add("patron_sucio", [pattern_sucio])

bigramas_sucios = []
for doc in nlp.pipe(prado_filtrado["textmining"], disable=["ner"]):
    matches = matcher_sucio(doc)
    if len(matches) > 0:
        for match_id, start, end in matches:
            # Extraemos el Lema en minúsculas, igual que en los apuntes
            bigramas_sucios.append(doc[start:end].lemma_.lower())

# Reutilizamos tu función terminos_top para ver la tabla
df_sucios = terminos_top(bigramas_sucios)

print("Top 30 Bigramas (CON RUIDO) para detectar stopwords:")
```

```
## Top 30 Bigramas (CON RUIDO) para detectar stopwords:
```

[Hide](#)

```
print(df_sucios.head(30).to_string(index=False))
```



```
##          termino  freq
##          juan muñoz  67
##          paolo veronese  65
##          antonio raphael  44
##          raphael mungs  44
##          gótico mediterráneo  40
##          visitas guiada  37
##          reina isabel  36
## conferencia impartido  35
##          visita virtual  34
##          siglo xxi  31
##          visita guiada  31
## patrocinio exclusivo  21
##          miguel falomir  21
##          obra maestro  20
##          siglo xix  20
## prado multiplicado  18
## memoria compartido  18
##          siglo xviii  18
##          femenino iii  17
##          último día  16
## altísima resolución  16
##          felipe iv  14
##          muñoz degrain  14
##          ciclo dedicado  14
## vista panorámicas  13
##          accede gratis  13
## veronese 1528-1588  12
##          santa cruz  12
##          maria dal  12
##          dal pozzolo  12
```

Basado en los resultados, vemos ciertos bigramas que no son relevantes, por lo que ahora realizamos un filtrado de estas palabras

[Hide](#)

```
import pandas as pd
from spacy.matcher import Matcher

# 1. Tu lista empírica de stopwords basada en la observación de tuits
stopwords = ["último", "día", "vista", "visita", "panorámica", "accede", "gratis", "impartido", "dedicado"]

matcher_limpio = Matcher(nlp.vocab)

# 2. El patrón matemático oficial del curso, ahora con TUS stopwords
pattern_limpio = [
    {"POS": {"IN": ["ADJ", "NOUN", "PROPN"]}, "LEMMA": {"NOT_IN": stopwords}},
    {"POS": {"IN": ["ADJ", "NOUN", "PROPN"]}, "LEMMA": {"NOT_IN": stopwords}}
]
matcher_limpio.add("patron_limpio", [pattern_limpio])

bigramas_limpios = []
for doc in nlp.pipe(prado_filtrado["textmining"], disable=["ner"]):
    matches = matcher_limpio(doc)
    if len(matches) > 0:
        for match_id, start, end in matches:
            # Extraemos tokens en minúsculas
            bigramas_limpios.append(doc[start:end].text.lower())

# 3. Visualizamos el resultado final usando tu función
df_bigramas_final = terminos_top(bigramas_limpios)

print("Top 15 Bigramas (Limpios):")
```

Top 15 Bigramas (Limpios):

Hide

```
print(df_bigramas_final.head(15).to_string(index=False))
```

```
##          termino  freq
##      juan muñoz    67
##      paolo veronese  65
##      raphael mungs  44
##      antonio raphael 44
##  gótico mediterráneo 40
##      visitas guiadas  37
##      reina isabel    35
##      siglo xxi       31
##  patrocinio exclusivo 21
##      miguel falomir   21
##      siglo xix       20
##      obras maestras  19
##      prado multiplicado 18
##      memoria compartida 18
##      siglo xviii     17
```

ahora vemos los trigramas

Hide

```
# matcher.remove("patron_bi")
matcher=Matcher(nlp.vocab)
pattern_tri = [
    {"POS": {"IN": ["ADJ", "NOUN", "PROPN"]}, "LEMMA": {"NOT_IN": stopwords}},
    {"POS": {"IN": ["ADJ", "NOUN", "PROPN"]}, "LEMMA": {"NOT_IN": stopwords}},
    {"POS": {"IN": ["ADJ", "NOUN", "PROPN"]}, "LEMMA": {"NOT_IN": stopwords}}
]
matcher.add("patron_tri", [pattern_tri])

trigramas_extraidos = []
for doc in nlp.pipe(prado_filtrado["textmining"]):
    matches = matcher(doc)
    for match_id, start, end in matches:
        trigramas_extraidos.append(doc[start:end].text)

df_trigramas = terminos_top(trigramas_extraidos)
print("\nTop 10 Trigramas más frecuentes (Tokens):")
```


Top 10 Trigramas más frecuentes (Tokens):

Hide

```
print(df_trigramas.head(10).to_string(index=False))
```

```
##          termino  freq
##  Antonio Raphael Mungs  44
##  Antonio Muñoz Degrain  12
##  Paolo Veronese 1528-1588 12
##      Paula Mues Orts    12
##  pintor Antonio Muñoz   11
##  Università degli Studi 10
##      degli Studi di     10
##      Enrico Maria Dal    9
##      Maria Dal Pozzolo    9
##      Noelia García Pérez  9
```

Podemos ver algunos trigramas como “Università degli Studi” y “degli Studi di” que son ambas partes de la misma institución. Más adelante se realizará una limpieza donde se filtrarán los n-gramas que forman parte de otros.

Ahora vamos a ver los diferentes nombres compuestos

[Hide](#)

```
# 1. Limpiar el matcher anterior para evitar interferencias
matcher.remove("patron_tri")

# 2. Definir los patrones oficiales de nombres propios consecutivos (de 2 a 5 tokens)
pattern_2 = [{"POS": "PROPN"}, {"POS": "PROPN"}]
pattern_3 = [{"POS": "PROPN"}, {"POS": "PROPN"}, {"POS": "PROPN"}]
pattern_4 = [{"POS": "PROPN"}, {"POS": "PROPN"}, {"POS": "PROPN"}, {"POS": "PROPN"}]
pattern_5 = [{"POS": "PROPN"}, {"POS": "PROPN"}, {"POS": "PROPN"}, {"POS": "PROPN"}, {"POS": "PROPN"}]

matcher.add("2gramas", [pattern_2])
matcher.add("3gramas", [pattern_3])
matcher.add("4gramas", [pattern_4])
matcher.add("5gramas", [pattern_5])

nombres_compuestos = []

# 3. Procesar sobre la columna "texto" original para máxima precisión del etiquetador POS
for doc in nlp.pipe(prado_filtrado["textmining"], disable=["ner"]):
    matches = matcher(doc)
    if len(matches) > 0:
        for match_id, start, end in matches:
            # Almacenamos el texto original respetando sus mayúsculas
            nombres_compuestos.append(doc[start:end].text)

# 4. Calcular frecuencias utilizando tu función terminos_top
df_nombres_compuestos = terminos_top(nombres_compuestos)

# 5. Imprimir el Top 45 de nombres propios compuestos
print("Top 45 Nombres Propios Compuestos en el corpus (Sin filtrar redundancias):")
```

```
## Top 45 Nombres Propios Compuestos en el corpus (Sin filtrar redundancias):
```

[Hide](#)

```
print(df_nombres_compuestos.head(25).to_string(index=False))
```

```
##          termino  freq
##      Juan Muñoz    67
##      Paolo Veronese 65
## Antonio Raphael Mengers 44
##      Antonio Raphael 44
##      Raphael Mengers 44
##      Miguel Falomir 21
##      Felipe IV      14
##      Muñoz Degrain  14
##      accede gratis  13
##      Santa Cruz     12
##      Antonio Muñoz  12
## Antonio Muñoz Degrain 12
##      Paula Mues Orts 12
##      Paula Mues     12
##      Enrico Maria   12
##      Mues Orts      12
##      di Verona      11
##      Siglo XXI      11
## Fundación Amigos   11
##      Buen Retiro    11
##      Luca Giordano  11
##      Capilla Herrera 10
##      Studi di       10
##      Galería Central 10
##      Maria Dal      9
```

podemos reemplazar los terminos que forman parte de otros, como se menciona anteriormente

[Hide](#)

```
# 1. Lista para guardar los nombres verdaderamente únicos
unicos = []

# 2. Iteramos sobre los 200 nombres más frecuentes
for text in df_nombres_compuestos["termino"].head(200):
    # Buscamos en cuántos registros está contenido este texto
    indices = [i for i, s in enumerate(df_nombres_compuestos["termino"]) if text in s]

    # Si solo aparece 1 vez (es decir, solo hace match consigo mismo)
    if len(indices) == 1:
        unicos.append(text)

# 3. Filtramos el DataFrame para quedarnos solo con los Nombres Propios consolidados
df_nombres_unicos = df_nombres_compuestos[df_nombres_compuestos["termino"].isin(unicos)].reset_index(drop=True)

# 4. Imprimimos el resultado limpio
print("Top 25 Nombres Propios Compuestos (Limpios de redundancias):")
```

```
## Top 25 Nombres Propios Compuestos (Limpios de redundancias):
```

[Hide](#)

```
print(df_nombres_unicos.head(25).to_string(index=False))
```

```
##          termino  freq
##          Juan Muñoz    67
##          Paolo Veronese 65
##          Felipe IV     14
##          accede gratis  13
##          Santa Cruz    12
##          Antonio Muñoz Degrain 12
##          Paula Mues Orts 12
##          Buen Retiro    11
##          Luca Giordano  11
##          Capilla Herrera 10
##          Galería Central 10
##          Annibale Carracci 9
##          Enrico Maria Dal Pozzolo 9
##          Noelia García Pérez 9
##          Alejandro Vergara 8
##          Studi di Verona 8
##          Día Internacional 7
##          José Aparicio  7
##          Fundación Loewe 7
##          Mathias Énard  7
##          Museo Nacional  7
##          Salman Rushdie  7
##          Gestión Integral 7
##          Isabel Clara Eugenia 6
##          Astrit Schmidt-Burkhardt 6
```

Ahora si podemos observar como los ngramas que forman parte de otros han sido limpiados, y se detecta la universidad como Studi di Verona

Ahora vamos a analizar las colocaciones:

[Hide](#)

```
from nltk.collocations import BigramCollocationFinder, BigramAssocMeasures
from nltk.collocations import TrigramCollocationFinder, TrigramAssocMeasures
import pandas as pd

print("\n--- Búsqueda de Palabras Clave: Colocaciones (PMI) ---")
```

```
##
## --- Búsqueda de Palabras Clave: Colocaciones (PMI) ---
```

[Hide](#)

```
# 1. Extraemos los tokens secuenciales limpios (como hace el manual)
tokens_coloc = []
for doc in nlp.pipe(prado_filtrado["textmining"], disable=["ner"]):
    for token in doc:
        # Filtramos por categoría gramatical y eliminamos tus stopwords usando el Lema
        if token.pos_ in ["NOUN", "ADJ", "PROPN"]:
            if token.lemma_.lower() not in stopwords:
                tokens_coloc.append(token.text.lower())

finder_bi = BigramCollocationFinder.from_words(tokens_coloc)
# Aplicamos un filtro de frecuencia (mínimo 3) para evitar que el PMI
# nos dé valores altísimos por palabras raras que solo salen juntas 1 vez.
finder_bi.apply_freq_filter(3)

bigram_measures = BigramAssocMeasures()
pmi_bi = []

for i in finder_bi.score_ngrams(bigram_measures.pmi):
    pmi_bi.append({"Palabra_1": i[0][0], "Palabra_2": i[0][1], "PMI": i[1]})

df_coloc_bi = pd.DataFrame(pmi_bi)
print("Top 15 Colocaciones de Bigramas por PMI:")
```

```
## Top 15 Colocaciones de Bigramas por PMI:
```

Hide

```
print(df_coloc_bi.head(15).to_string(index=False))
```

```
##      Palabra_1  Palabra_2    PMI
##      apóstoles      simón 12.08359
##      carvajal      vargas 12.08359
## carvajal-vargas  sotomayor 12.08359
##      cerro      tepeyac 12.08359
##      conocer  polarizada 12.08359
##      elizabeth  hernández 12.08359
##      escuelas      norte 12.08359
##      estampas  fotografías 12.08359
##      esther      peces 12.08359
##      fragua      vulcano 12.08359
##      iara      solano 12.08359
##      irrepetible  profundidad 12.08359
##      medias      res 12.08359
##      movilidad      age 12.08359
##      nimpha      carvajal 12.08359
```

Ahora las colocaciones de trigramas

Hide

```
finder_tri = TrigramCollocationFinder.from_words(tokens_coloc)
finder_tri.apply_freq_filter(3)

trigram_measures = TrigramAssocMeasures()
pmi_tri = []

for i in finder_tri.score_ngrams(trigram_measures.pmi):
    pmi_tri.append({"Palabra_1": i[0][0], "Palabra_2": i[0][1], "Palabra_3": i[0][2], "PMI": i[1]})

df_coloc_tri = pd.DataFrame(pmi_tri)
print("\nTop 15 Colocaciones de Trigramas por PMI:")
```

```
##
## Top 15 Colocaciones de Trigramas por PMI:
```

Hide

```
print(df_coloc_tri.head(15).to_string(index=False))
```

```
##      Palabra_1  Palabra_2  Palabra_3    PMI
## carvajal-vargas  sotomayor    alarcón 24.167180
##      cerro      tepeyac      1531 24.167180
##      nimpha      carvajal    vargas 24.167180
##      religiosa  surgida      cerro 24.167180
##      revelada  objeto      culto 24.167180
##      surgida      cerro      tepeyac 24.167180
##      1531  fronteras novohispanas 23.752143
##      apóstoles      simón      mateo 23.752143
##      conocer  polarizada    fortuna 23.752143
##      dibujos  estampas  fotografías 23.752143
##      fascinante  tabla    manierista 23.752143
##      natural      pagano    cristiano 23.752143
##      ocasión irrepetible  profundidad 23.752143
##      simón      mateo      catedral 23.752143
##      tepeyac      1531    fronteras 23.752143
```

También podemos ver los términos más significativos mediante el likelihood

Hide

```
llr_tri = []
for i in finder_tri.score_ngrams(trigram_measures.likelihood_ratio):
    llr_tri.append({"Palabra_1": i[0][0], "Palabra_2": i[0][1], "Palabra_3": i[0][2], "LLR": i[1]})
df_coloc_tri_llr = pd.DataFrame(llr_tri)
```

```
print("\nTop 15 por Likelihood Ratio (Términos Estables):")
```

```
##
## Top 15 por Likelihood Ratio (Términos Estables):
```

Hide

```
print(df_coloc_tri_llr.head(15).to_string(index=False))
```

```
## Palabra_1 Palabra_2 Palabra_3 LLR
## museo prado siglo 2933.211111
## amigos museo prado 2920.180165
## septiembre museo prado 2913.532697
## museo prado exposiciones 2899.286938
## museo prado panorámicas 2887.217906
## virtual museo prado 2886.525969
## director museo prado 2884.160220
## colecciones museo prado 2878.718877
## museo prado patrocinio 2854.164305
## auditorio museo prado 2846.485241
## marzo museo prado 2843.735574
## xvii museo prado 2841.894601
## museo prado enrico 2839.592333
## museo prado colaboración 2838.649085
## museo prado detalles 2838.422492
```

Vemos que los terminos relativos al museo del prado son los dominantes. con un peso notablemente alto

2.1.3 Visualizar las coocurrencias.

Dibujamos las relaciones y fuerzas de enlace entre los términos que aparecen juntos con mayor frecuencia dentro del mismo tuit.

Hide

```

from nltk.collocations import BigramCollocationFinder, BigramAssocMeasures
import pandas as pd
lemas_cooc = []
for doc in nlp.pipe(prado_filtrado["textmining"], disable=["ner"]):
    for token in doc:
        if token.pos_ in ["NOUN", "ADJ", "PROPN"]:
            if token.lemma_.lower() not in stopwords:
                lemas_cooc.append(token.lemma_.lower())

finder_lemas = BigramCollocationFinder.from_words(lemas_cooc)

# 3. Aplicamos un filtro de frecuencia (frecuencia mínima = 3 para redes sociales)
finder_lemas.apply_freq_filter(3)

# 4. Usamos la medida de frecuencia bruta (raw_freq) para evaluar coocurrencias
bigram_measures = BigramAssocMeasures()

left = []
right = []
freq = []

for i in finder_lemas.score_ngrams(bigram_measures.raw_freq):
    left.append(i[0][0]) # Palabra 1 (String)
    right.append(i[0][1]) # Palabra 2 (String)
    freq.append(i[1]) # Frecuencia bruta (Float)

# 5. Generamos el DataFrame de coocurrencias
df_cooc = pd.DataFrame({
    "left": left,
    "right": right,
    "freq": freq
})

# 6. Limpieza: Evitamos que una palabra coocurra consigo misma (lazos circulares)
df_cooc = df_cooc[df_cooc["left"] != df_cooc["right"]]

# 7. Multiplicamos la frecuencia por 10.000 (el truco matemático del profesor)
# Esto convierte la probabilidad decimal en una "frecuencia esperada por cada 10k palabras",
# ideal para usarse después como grosor físico de las líneas del grafo.
df_cooc["freq"] = df_cooc["freq"] * 10000

print("Top 15 Coocurrencias de Lemas más frecuentes:")

```

```
## Top 15 Coocurrencias de Lemas más frecuentes:
```

[Hide](#)

```
print(df_cooc.head(15).to_string(index=False))
```


##	left	right	freq
##	museo	prado	381.691114
##	isabel	farnesio	55.295292
##	juan	muñoz	51.455341
##	paolo	veronese	49.919361
##	prado	femenino	49.151371
##	veronese	1528-1588	46.847400
##	guadalupe	méxico	38.399508
##	historias	arte	38.399508
##	muñoz	historias	38.399508
##	méxico	españa	36.095538
##	antonio	raphael	33.791567
##	raphael	mengs	33.791567
##	manera	italia	33.023577
##	italia	españa	32.255587
##	españa	gótico	30.719607

[Hide](#)

```
import networkx as nx
import matplotlib.pyplot as plt

# Configuramos un lienzo amplio para evitar apilamientos
plt.figure(figsize=(11, 11))

# Tomamos las 40 coocurrencias más fuertes para un gráfico limpio y legible
df_nx = df_cooc.head(40)

# Creamos el grafo
G = nx.from_pandas_edgelist(
    df_nx,
    source="left",
    target="right",
    edge_attr="freq",
    create_using=nx.Graph()
)

# Posicionamiento armónico de nodos (aumentamos 'k' para separar las palabras)
pos = nx.spring_layout(G, k=0.45, seed=42)

# Calculamos grosores de línea proporcionales a la fuerza de coocurrencia
edges = G.edges(data=True)
weights = [d["freq"] for (_, _, d) in edges]
max_w = max(weights) if weights else 1
# Escalamos los grosores entre 1 y 6 para que se vean bien las líneas
norm_weights = [1 + 5 * (w / max_w) for w in weights]

# A) Pintamos las uniones (aristas) de forma muy clara en gris azulado translúcido
nx.draw_networkx_edges(
    G, pos,
    width=norm_weights,
    alpha=0.4,
    edge_color="steelblue"
)

# B) Pintamos los nodos como círculos discretos de color azul pastel
nx.draw_networkx_nodes(
    G, pos,
    node_color="#add8e6",
    node_size=180,
    edgecolors="gray",
    linewidths=0.5
)

# C) Pintamos las etiquetas de texto con un ligero desplazamiento para que no tapen el nodo
# Añadimos una tipografía elegante en verde oscuro
nx.draw_networkx_labels(
    G, pos,
    font_color="#1b4332",
    font_size=8,
    font_weight="bold"
)
```

```
## {'museo': Text(-0.14677257394394733, 0.21329334444326073, 'museo'), 'prado': Text(0.26260973861386594, 0.1349740917269198, 'prado'), 'isabel': Text(-0.8967611808902822, -0.1516222425053222, 'isabel'), 'farnesio': Text(-0.7695460112625018, 0.07640648904048038, 'farnesio'), 'juan': Text(-0.020858545245383536, 0.8844719594859325, 'juan'), 'muñoz': Text(-0.2691135239459477, 0.9480522307017822, 'muñoz'), 'paolo': Text(0.6519609524976477, -0.6450977686186057, 'paolo'), 'veronese': Text(0.3428461043330362, -0.7050224441859453, 'veronese'), 'femenino': Text(0.4739167636045613, 0.07763451111867206, 'femenino'), '1528-1588': Text(0.21703265420242615, -0.862010289474937, '1528-1588'), 'guadalupe': Text(-0.38659939314683756, -0.5319887987035146, 'guadalupe'), 'méxico': Text(-0.5921890631129336, -0.6691127882382986, 'méxico'), 'historias': Text(-0.4390941530553636, 0.7052183353393419, 'historias'), 'arte': Text(-0.6438340504651321, 0.5426960997338434, 'arte'), 'españa': Text(-0.5103287508107284, -0.5645914412722182, 'españa'), 'antonio': Text(0.5053961437915362, 0.3330226929771829, 'antonio'), 'raphael': Text(0.38315323909140425, 0.5574154649283508, 'raphael'), 'mengs': Text(0.1748518415454375, 0.7410778136028378, 'mengs'), 'manera': Text(-0.1993117483702463, -0.9660651813743394, 'manera'), 'italia': Text(-0.14091152610819374, -0.6423714365327147, 'italia'), 'gótico': Text(-0.7900933804764079, -0.21656743398621325, 'gótico'), 'mediterráneo': Text(-0.8706855198670785, 0.21391965077640343, 'mediterráneo'), '1320-1420': Text(-0.6194159974359594, 0.5611214989271059, '1320-1420'), 'reina': Text(-0.6193033051995209, -0.23091131131175552, 'reina'), 'exposición': Text(0.7545634298198085, -0.29984578430544645, 'exposición'), 'visitas': Text(0.9960005228527892, 0.2385760013625757, 'visitas'), 'guiada': Text(0.8900972769917025, 0.052703932456938996, 'guiada'), 'vídeo': Text(-0.39109981243094744, 0.7584395978311704, 'vídeo'), 'conferencia': Text(-0.4666067650689521, 0.4883420971653025, 'conferencia'), '1728-1779': Text(-0.019944100932819394, 0.4855439954395259, '1728-1779'), 'colección': Text(0.22662644841558915, 0.01524725017918898, 'colección'), 'siglo': Text(0.5679148176700352, 0.25482325779816994, 'siglo'), 'xxi': Text(0.5053924028889347, 0.39016096078969054, 'xxi'), 'tan': Text(-0.413849111083374, -0.3819823605008663, 'tan'), 'septiembre': Text(-0.22813148886681497, 0.5092829172093295, 'septiembre'), 'exposiciones': Text(0.31761695863104705, -0.15385215051982076, 'exposiciones'), 'taquilla': Text(-0.5409793992650946, 0.31779253546277475, 'taquilla'), 'entradas': Text(0.7299425747106016, -0.22047211206448783, 'entradas'), 'horario': Text(0.8521238981100371, -0.4896981366261405, 'horario'), 'obra': Text(0.6551870167864366, -0.09158045114351318, 'obra'), 'artista': Text(0.9773431527388118, 0.035323228451454955, 'artista'), 'español': Text(-0.1050010698850016, -1.0, 'español'), 'inglés': Text(-0.286638536517796, -0.8499869176887977, 'inglés'), '1692-1766': Text(-0.44402113230200824, 0.2550308488576398, '1692-1766'), 'información': Text(0.9943333641775801, -0.022515144095197254, 'información'), 'miguel': Text(-0.032757051358512056, 0.6584309120587369, 'miguel'), 'falomir': Text(0.20906122318947867, 0.695537104254727, 'falomir'), 'maestro': Text(0.8707870495734724, -0.33242217779673866, 'maestro'), 'patrocinio': Text(-0.5082110497144637, -0.5254379880901916, 'patrocinio'), 'exclusivo': Text(-0.6933949660783408, -0.774112756864596, 'exclusivo'), 'virtual': Text(-0.5133043673956492, 0.18272829378032138, 'virtual')}
```

Hide

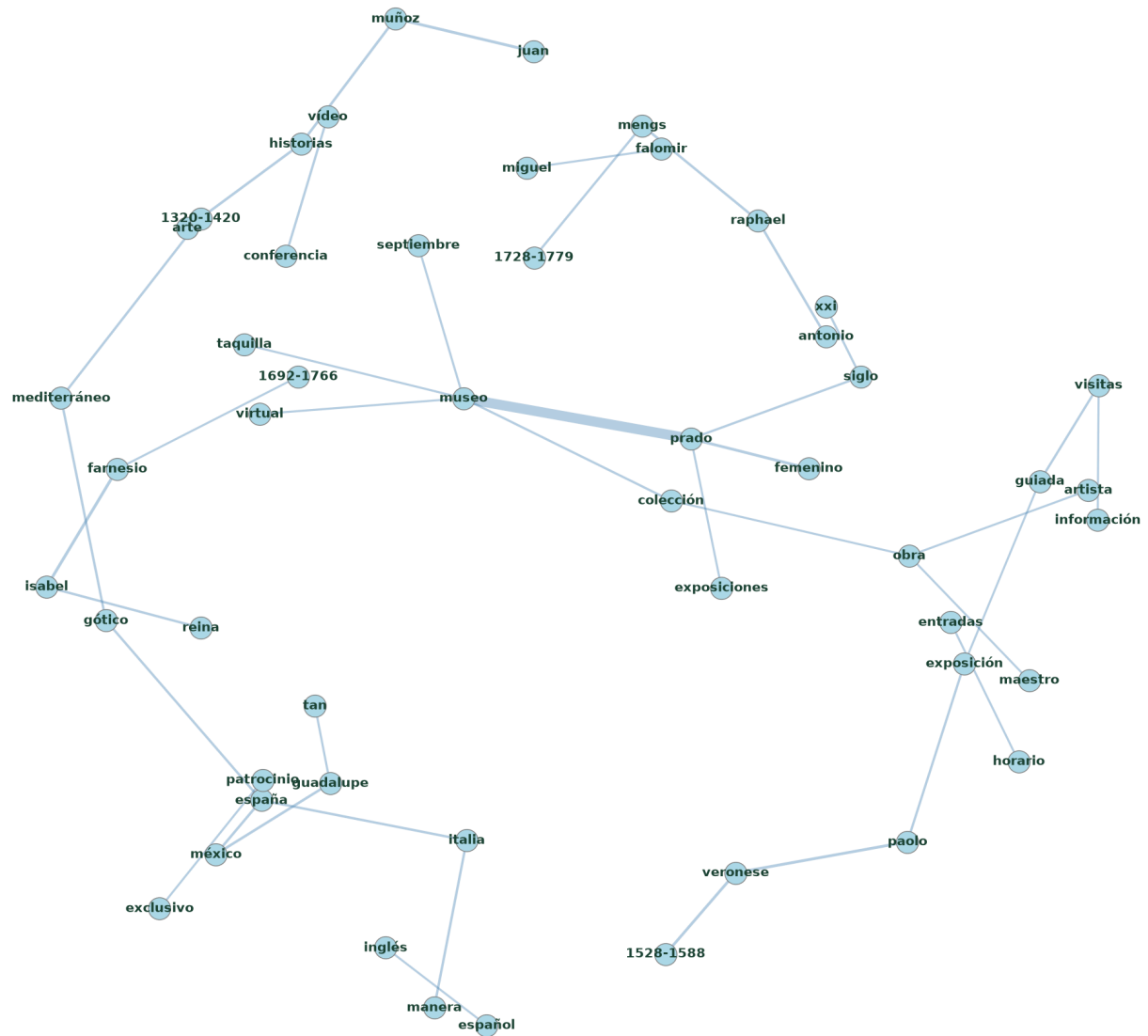
```
plt.title("Red Semántica de Coocurrencias del Museo del Prado\n(Top 40 relaciones más frecuentes)",
         fontsize=13, fontweight="bold")
plt.axis("off")
```

```
## (np.float64(-1.0955011597833046), np.float64(1.1947405017458117), np.float64(-1.204545484223687), np.float64(1.1525977149254691))
```

Hide

```
plt.tight_layout()
plt.show()
```

Red Semántica de Coocurrencias del Museo del Prado (Top 40 relaciones más frecuentes)



Podemos observar un nucleo tematico entrono a la fuerte conexion de *museo-prado* de donde podemos ver que salen conceptos relacionados con la institución, como *exposiciones*, *taquilla*, *virtual*.

Podemos ver otros pequeños subsistemas temáticos en torno a *paolo-veronese*, *juan-muñoz*, *españa-mexico* ligado a diferentes regiones geograficas, y un subsistema ligado a figuras femeninas en torno a *isabel-reina*

Tambien podemos crear un cluster interactivo en HTML para poder observar con mas detalle la red

[Hide](#)

```

from pyvis.network import Network
from sklearn.preprocessing import MinMaxScaler

# Seleccionamos las 100 relaciones más fuertes para la red interactiva
df_pyvis = df_cooc.head(100).copy()

# Escalamos el grosor de las líneas interactivas
scaler = MinMaxScaler(feature_range=(1, 7))
df_pyvis["width"] = scaler.fit_transform(df_pyvis[["freq"]])

# Inicializamos la red de PyVis
net = Network(height="600px", width="100%", bgcolor="#ffffff", font_color="#333333")

# Llenamos la red interactiva evitando duplicar nodos
nodos_agregados = set()
for index, row in df_pyvis.iterrows():
    u, v, w = row["left"], row["right"], row["width"]

    if u not in nodos_agregados:
        net.add_node(u, label=u, title=u, color="#8ecae6")
        nodos_agregados.add(u)
    if v not in nodos_agregados:
        net.add_node(v, label=v, title=v, color="#8ecae6")
        nodos_agregados.add(v)

    net.add_edge(u, v, width=w, color="lightgray")

# Configuramos opciones de física interactiva agradables para el usuario
net.set_options("""
{
  "physics": {
    "enabled": true,
    "barnesHut": {
      "gravitationalConstant": -1500,
      "centralGravity": 0.3,
      "springLength": 95,
      "springConstant": 0.04
    },
    "stabilization": {"iterations": 500}
  },
  "nodes": {
    "shape": "dot",
    "size": 10,
    "font": {"size": 12, "face": "arial"}
  },
  "edges": {
    "color": {"color": "lightgray", "highlight": "orange"},
    "arrows": {"to": {"enabled": false}},
    "smooth": {"type": "continuous"}
  }
}
""")

# Guardamos el archivo HTML interactivo
net.show("red_interactiva_prado.html", notebook=False)

```

```
## red_interactiva_prado.html
```

[Hide](#)

```
print("\n¡Grafo interactivo generado con éxito en 'red_interactiva_prado.html'!")
```

```
##
## ¡Grafo interactivo generado con éxito en 'red_interactiva_prado.html'!
```

El análisis visual de la Red de Coocurrencias revela que el núcleo de la red está dominado por los conceptos de la entidad ('museo', 'prado', 'visita guiada'). Sin embargo, en torno a este núcleo, aparecen tres agrupaciones temáticas: en primer lugar, el cluster cerrado que rodea a 'Paolo Veronese (1528-1588)'; en segundo lugar, un cluster que vincula a 'Isabel' (Farnesio) con la perspectiva femenina; y finalmente, un nodo aislado y sumamente específico dedicado a la obra escultórica de 'Juan Muñoz'. Este mapa refleja que las redes sociales del museo combinan de manera equilibrada la agenda diaria con grandes hitos de divulgación histórica y contemporánea.

2.1.4 Reconocimiento de entidades nombradas (NER).

Utilizamos el reconocedor de entidades de spaCy para extraer nombres de personas, ubicaciones y organizaciones de los tuits.

Hide

```
import pandas as pd
from collections import Counter

# 1. Inicializamos listas para guardar las entidades según su categoría
personas = []
lugares = []
organizaciones = []
miscelanea = []

# 2. Procesamos el corpus con nlp.pipe (¡MANTENIENDO EL NER ACTIVO!)
# Usamos la columna "texto" original para que spaCy mantenga las mayúsculas de pista
for doc in nlp.pipe(prado_filtrado["textmining"]):
    for ent in doc.ents:
        texto_entidad = ent.text.strip()

        # Filtramos entidades de longitud muy corta o vacías por seguridad
        if len(texto_entidad) < 2:
            continue

        # Clasificamos según la etiqueta que le asigna spaCy (.label_)
        if ent.label_ == "PER":
            personas.append(texto_entidad)
        elif ent.label_ == "LOC":
            # El profesor unifica el Museo del Prado bajo la misma etiqueta LOC
            if "Prado" in texto_entidad or "Museo del Prado" in texto_entidad:
                lugares.append("Museo del Prado")
            else:
                lugares.append(texto_entidad)
        elif ent.label_ == "ORG":
            organizaciones.append(texto_entidad)
        elif ent.label_ == "MISC":
            miscelanea.append(texto_entidad)

# 3. Creamos tablas de frecuencia usando la función auxiliar del profesor
df_personas = terminos_top(personas)
df_lugares = terminos_top(lugares)
df_organizaciones = terminos_top(organizaciones)
df_miscelanea = terminos_top(miscelanea)

# 4. Mostramos los resultados de las categorías principales para el informe
print("=== TOP 15 PERSONAS DETECTADAS (PER) ===")
```

```
## === TOP 15 PERSONAS DETECTADAS (PER) ===
```

Hide

```
print(df_personas.head(15).to_string(index=False))
```

```
##          termino  freq
##      Paolo Veronese    65
##      Juan Muñoz      65
##      España          49
##      Isabel de Farnesio 49
## Antonio Raphael Mengs 41
##      Goya            41
##      Velázquez       33
##      Visitas         30
##      Rubens          20
##      Estés           19
##      Más             19
##      Jueves          19
##      Miguel Falomir   18
##      Renacimiento     18
##      Martes          18
```

Hide

```
print("\n=== TOP 15 LUGARES DETECTADOS (LOC) ===")
```

```
##
## === TOP 15 LUGARES DETECTADOS (LOC) ===
```

Hide

```
print(df_lugares.head(15).to_string(index=False))
```

```
##          termino  freq
##      Museo del Prado 627
##      España          58
##      Guadalupe de México 48
##      Italia          42
##      Celebrada       30
##      Madrid         16
##      Entradas        14
##      museo\n Todas   14
##      Roma           12
##      Visitas         11
##      Capilla Herrera 10
##      Guadalupe       10
##      Sábado          9
##      Salas           8
##      Qué             8
```

Hide

```
print("\n=== TOP 15 ORGANIZACIONES DETECTADAS (ORG) ===")
```

```
##
## === TOP 15 ORGANIZACIONES DETECTADAS (ORG) ===
```

Hide

```
print(df_organizaciones.head(15).to_string(index=False))
```

##	termino	freq
##	Neoclasicismo	13
##	Consulta	10
##	Colabora	9
##	Studi di Verona	8
##	Historia	7
##	Programa	7
##	Fundación Amigos del Museo del Prado	6
##	Programa de Gestión Integral de Museos del Museo del Prado	6
##	Día Internacional de los Museos	4
##	Veronese	4
##	Galería Central	4
##	Consejo de Estado	4
##	Itinerario El	3
##	Agencia	3
##	Universidad de Sevilla	2

De nuevo vemos que el Museo del Prado es la entidad con mayor frecuencia. Sin embargo, el modelo trata “Museo del Prado”, “El Prado” y “Prado” como entidades distintas, lo que muestra las limitaciones del modelo de lenguaje natural.

España sale clasificada como persona, esto puede deberse a la falta de la correcta sintaxis en los tuits donde se menciona a España, denominandola de la misma manera que se haría con una persona, con mayúsculas. Lo mismo puede ocurrir para “Visitas”
Celebrada aparece como localización .

Estos hechos evidencian las limitaciones de los modleos pequeños para el correcto analisis de tuits, donde no se siguen las reglas del lenguaje formal

Gráfico de Entidades Nombradas por Categoría

Hide


```

import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

# =====
# 1. ADAPTACIÓN DE TUS VARIABLES (EL PUENTE)
# =====
# Añadimos la etiqueta de tipo a las tablas que generaste antes
df_personas['Tipo'] = 'PER'
df_lugares['Tipo'] = 'LOC'
df_organizaciones['Tipo'] = 'ORG'

# Unimos (concatenamos) las tres tablas en una sola
df_todas = pd.concat([df_personas, df_lugares, df_organizaciones])

# Ordenamos de mayor a menor frecuencia y sacamos el Top 15 global
top_entidades = df_todas.sort_values(by="freq", ascending=False).head(15).copy()

# Renombramos las columnas para que encajen exactamente con tu código de la gráfica
top_entidades.rename(columns={"termino": "Entidad", "freq": "Frecuencia"}, inplace=True)

# =====
# 2. TU CÓDIGO DE VISUALIZACIÓN
# =====
# Crear lienzo explícito
fig, ax = plt.subplots(figsize=(10, 6))

# Configurar mapa de colores según el tipo de entidad
colores_dict = {"PER": "salmon", "LOC": "skyblue", "ORG": "lightgreen"}
colores = [colores_dict.get(tipo, "gray") for tipo in top_entidades["Tipo"]]

# Dibujar barras horizontales
ax.barh(top_entidades["Entidad"], top_entidades["Frecuencia"], color=colores)
ax.invert_yaxis()
ax.set_xlabel("Frecuencia")
ax.set_ylabel("Entidades")

# Título limpio (sin texto de Leyenda)
ax.set_title("Top 15 Entidades Nombradas más frecuentes", fontsize=14, weight="bold")

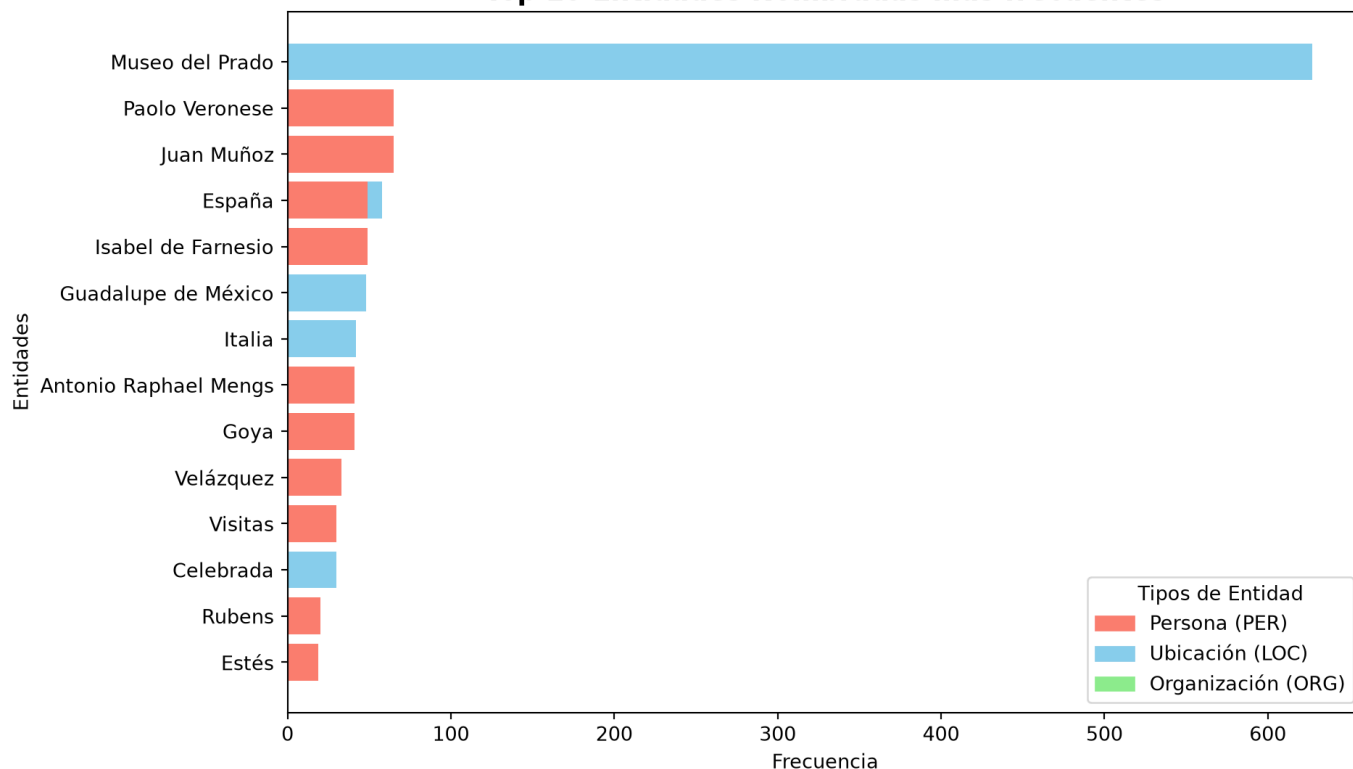
# Crear los elementos visuales de la leyenda de forma manual
parches_leyenda = [
    mpatches.Patch(color="salmon", label="Persona (PER)"),
    mpatches.Patch(color="skyblue", label="Ubicación (LOC)"),
    mpatches.Patch(color="lightgreen", label="Organización (ORG)")
]

# Añadir la leyenda al gráfico
ax.legend(handles=parches_leyenda, loc="lower right", title="Tipos de Entidad", frameon=True, facecolor="white", edgecolor="lightgray")

plt.tight_layout()
plt.show()

```

Top 15 Entidades Nombradas más frecuentes



2.1.5 Análisis de Casos Específicos

Podemos clasificar los tuits conforme a una temática determinada. Para extraer temas, podemos basarnos en la información que se desprende del text mining. Se pide:

- Examinando las palabras clave (bigramas, trigramas, etc) y las coocurrencias obtenidas, indica algunos nombres de pintores y exposiciones.

Si te resulta más cómodo, señala las relaciones oportunas (término1 – término2 – término3 – término4).

Respecto a los pintores, pudimos identificar varios de ellos en el text mining anterior, como *Juan Muñoz*, *Pablo Veronese*, aunque la identificación de las exposiciones no fue tan clara. Para facilitarlo, podemos buscar los n-gramas relacionados con exposición o exposiciones

[Hide](#)

```
df_bi_expo = df_bigramas_final[df_bigramas_final["termino"].str.contains("exposici", case=False, na=False)]

print("--- Bigramas que contienen 'exposición' o 'exposiciones': ---")
```

```
## --- Bigramas que contienen 'exposición' o 'exposiciones': ---
```

[Hide](#)

```
print(df_bi_expo.head(10).to_string(index=False))
```

```
##          termino  freq
## exposición comisariada    8
##      exposición paolo    7
## exposiciones temporales    6
##      nueva exposición    6
##      exposición prado    4
##      exposiciones paolo    4
##      exposición homenaje    2
##      exposición didáctica    2
##      gran exposición    2
##      exposición valeriano    1
```

[Hide](#)

```
# 2. Filtrar en tus Trigramas Limpios
df_tri_expo = df_trigramas[df_trigramas["termino"].str.contains("exposici", case=False, na=False)]

print("\n--- Trigramas que contienen 'exposición' o 'exposiciones': ---")
```

```
##
## --- Trigramas que contienen 'exposición' o 'exposiciones': ---
```

[Hide](#)

```
print(df_tri_expo.head(10).to_string(index=False))
```

```
##          termino  freq
##  exposición Paolo Veronese    7
##  exposiciones Paolo Veronese    4
##  exposición Prado Siglo      2
##  próxima exposición Prado    1
##  exposición Juan Muñoz       1
##  exposición Antonio Raphael   1
##  primera gran exposición      1
##  nueva exposición homenaje    1
##  exposición Valeriano Bécquer  1
```

- Selecciona aquellos tuits donde figure la cadena de texto "Veronese" . Después, muestra el texto de los cinco primeros tuits (ordenados por fecha, del tuit más antiguo al más reciente). Para finalizar, visualiza las coocurrencias.
- Ídem a la pregunta anterior, pero empleando la cadena de texto "femenino" .

Definimos una función para analizar ambas palabras clave y mostramos los tuits en orden cronológico

[Hide](#)

```

import pandas as pd
import networkx as nx
import itertools
from collections import Counter
import matplotlib.pyplot as plt

# Tu lista de stopwords validada empíricamente
stopwords = ["último", "día", "vista", "visita", "panorámica", "accede", "gratis", "impartido", "dedicado"]

def analizar_caso_especifico(palabra_clave, df, nlp_model):
    print(f"\n" + "="*50)
    print(f"ANÁLISIS ESPECÍFICO: '{palabra_clave.upper()}'")
    print("="*50)

    # 1. Filtramos el DataFrame y ordenamos por fecha (del más antiguo al más reciente)
    df_filtrado = df[df["textmining"].str.contains(palabra_clave, case=False, na=False)].copy()
    df_filtrado = df_filtrado.sort_values(by="fecha", ascending=True)

    print(f"Total de tuits encontrados: {len(df_filtrado)}\n")
    print("--- TOP 5 TUIITS MÁS ANTIGUOS ---")

    # 2. Imprimimos los 5 primeros tuits
    for idx, row in df_filtrado.head(5).reset_index().iterrows():
        # Formateamos la fecha para que quede elegante
        fecha_str = row['fecha'].strftime('%Y-%m-%d')
        print(f"{idx+1}. [{fecha_str}] {row['texto']}")
        print("-" * 30)

    # 3. Calculamos coocurrencias locales
    cooc_local = Counter()
    for doc in nlp_model.pipe(df_filtrado["textmining"], disable=["ner"]):
        lemas_unicos = list(set([
            token.lemma_.lower() for token in doc
            if token.pos_ in ["NOUN", "ADJ", "PROPN"]
            and token.lemma_.lower() not in stopwords
        ]))

        for p1, p2 in itertools.combinations(sorted(lemas_unicos), 2):
            cooc_local[(p1, p2)] += 1

    # Si no hay suficientes datos para un grafo, salimos
    if not cooc_local:
        print("No hay suficientes relaciones para dibujar un grafo.")
        return

    # 4. Preparamos los datos para NetworkX (Top 15 relaciones locales)
    df_cooc = pd.DataFrame([{"nodo1": k[0], "nodo2": k[1], "peso": v} for k, v in cooc_local.items()])
    top_cooc = df_cooc.sort_values(by="peso", ascending=False).head(15)

    # 5. Dibujamos el mini-grafo
    G = nx.from_pandas_edgelist(top_cooc, source="nodo1", target="nodo2", edge_attr="peso", create_using=nx.Graph())

    plt.figure(figsize=(8, 6))
    # Ajustamos 'k' para que los nodos no colapsen
    pos = nx.spring_layout(G, k=0.8, seed=42)

    pesos = [G[u][v]['peso'] for u,v in G.edges()]
    max_peso = max(pesos) if pesos else 1
    grososres = [(w / max_peso) * 4 for w in pesos]

    nx.draw_networkx_edges(G, pos, width=grososres, edge_color="gray", alpha=0.6)
    nx.draw_networkx_nodes(G, pos, node_color="#ff9999", node_size=800, edgecolors="white")
    nx.draw_networkx_labels(G, pos, font_size=10, font_weight="bold", font_color="black")

    plt.title(f"Red Local de Coocurrencias: '{palabra_clave}'", fontsize=14, fontweight="bold")
    plt.axis("off")

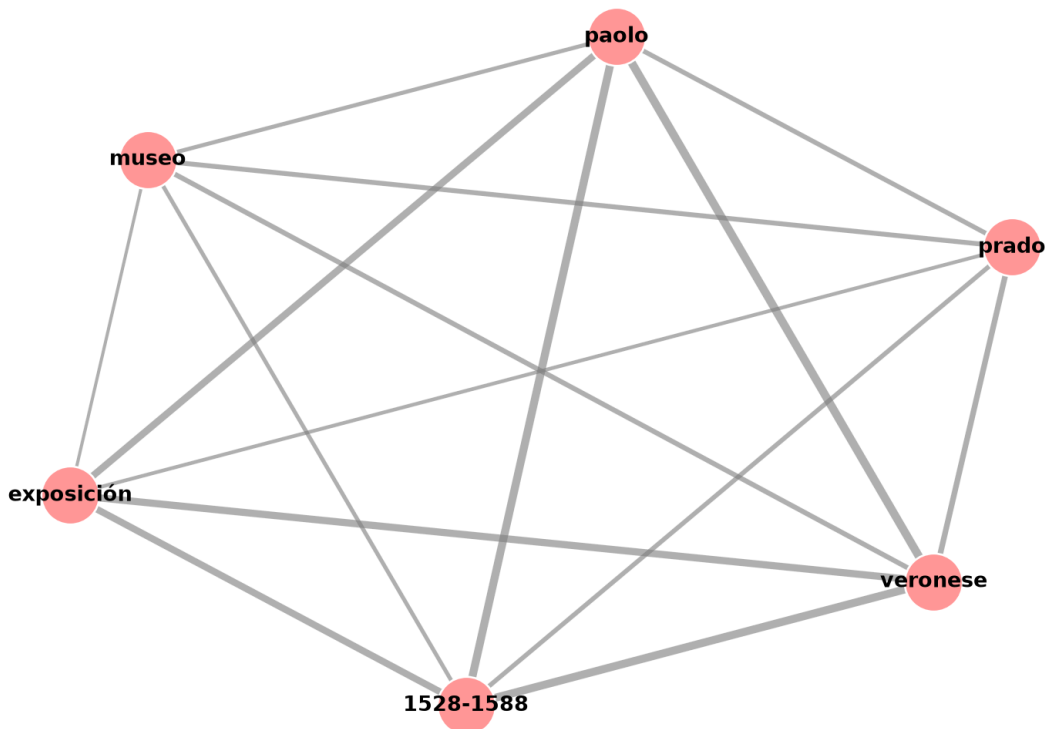
```

```
plt.tight_layout()
plt.show()
```

```
# Ejecutamos la función para los dos casos que pide el examen
analizar_caso_especifico("Veronese", prado_filtrado, nlp)
```

```
##
## =====
## ANÁLISIS ESPECÍFICO: 'VERONESE'
## =====
## Total de tuits encontrados: 71
##
## --- TOP 5 TUIITS MÁS ANTIGUOS ---
## 1. [2025-06-04] "Paolo Veronese 1528-1588", catálogo de la exposición comisariada por Miguel Falomir, director
del Museo del Prado, y Enrico Maria dal Pozzolo, de la Università degli Studi di Verona. Veronés cierra el ciclo d
edicado a la pintura veneciana del Renacimiento https://t.co/yABclXyrvG https://t.co/gRqGC1t88I
## -----
## 2. [2025-06-04] #Visitaguiada con Miguel Falomir, comisario de la exposición "Paolo Veronese (1528-1588)". Los
días 10, 11, 23 y 24 de junio. Inscripciones abiertas: https://t.co/WVdCILsAHX https://t.co/Jbx9XU9aex
## -----
## 3. [2025-06-05] #Visitasguiadas a la exposición "Paolo Veronese (1528-1588)". De martes a domingo a las 12:00 h
https://t.co/bz8apRw60D https://t.co/G4CXS10k2A
## -----
## 4. [2025-06-06] El Museo del Prado abre en horario nocturno la exposición "Paolo Veronese (1528-1588)" este sáb
ado #ElPradodeNoche https://t.co/P5RwoHL553 https://t.co/E2D9y90hSk
## -----
## 5. [2025-06-06] #ElPradodeNoche Este sábado 7 podrán visitarse gratuitamente en horario nocturno las exposicion
es "Paolo Veronese (1528-1588)" y "Cambio de forma. Mito y metamorfosis en los dibujos romanos de José de Madrazo"
https://t.co/XYlgn3DFf
## Colaboran @radio3_rne y @SamsungEspana
## -----
```

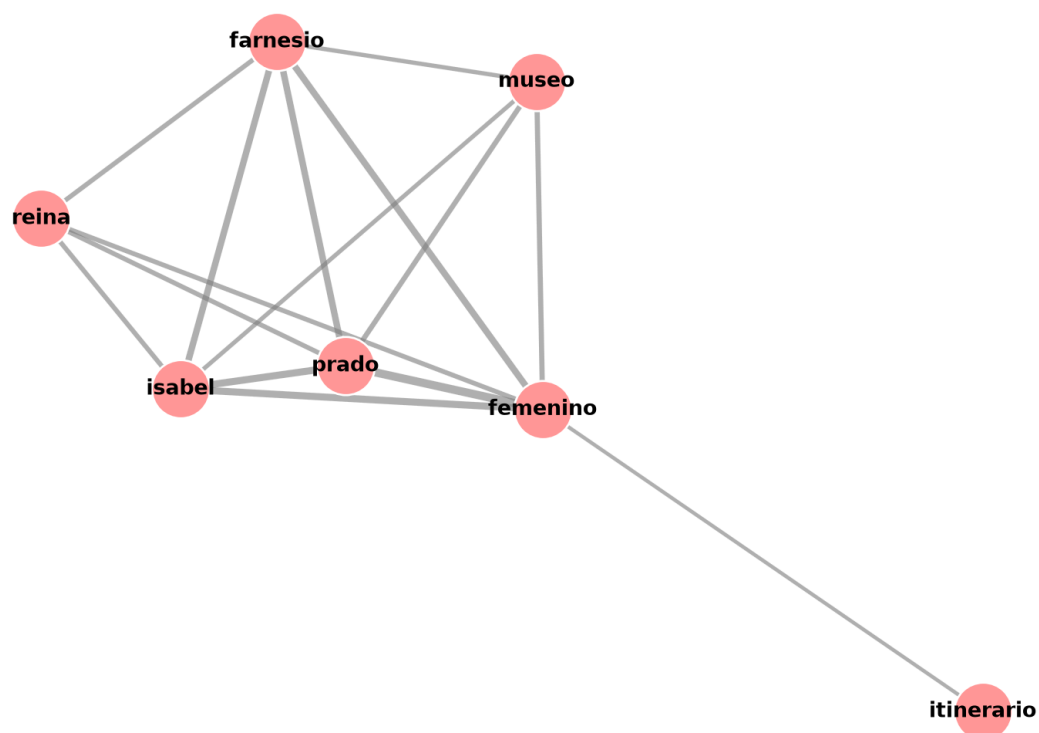
Red Local de Coocurrencias: 'Veronese'


[Hide](#)

```
analizar_caso_especifico("femenino", prado_filtrado, nlp)
```

```
##
## =====
## ANÁLISIS ESPECÍFICO: 'FEMENINO'
## =====
## Total de tuits encontrados: 57
##
## --- TOP 5 TUIITS MÁS ANTIGUOS ---
## 1. [2025-08-08] Serie documental "El Prado en femenino". Reinas, princesas, regentes y gobernadoras que contribuyeron poderosamente tanto a la creación y consolidación del Museo del Prado como a enriquecer sus colecciones https://t.co/KnUjNvnA3G https://t.co/91qf8pcDMz
## -----
## 2. [2025-08-26] La serie documental "El Prado en femenino" cuenta con episodios dedicados a Isabel la Católica, María de Hungría, Isabel de Valois, Isabel Clara Eugenia, Isabel de Borbón, Cristina de Suecia o Mariana de Austria. Disponible en YouTube https://t.co/Dr8V8DFXbE https://t.co/iPrAffYRnw
## -----
## 3. [2025-09-01] La serie documental "El Prado en femenino" cuenta con episodios dedicados a Isabel la Católica, María de Hungría, Isabel de Valois, Isabel Clara Eugenia, Isabel de Borbón, Cristina de Suecia o Mariana de Austria. Disponible en YouTube https://t.co/p490QbSs5W #ElPradoEnFemenino https://t.co/2lz5hYNI82
## -----
## 4. [2025-09-10] Serie documental "El Prado en femenino", con episodios dedicados a Isabel la Católica, María de Hungría, Isabel de Valois, Isabel Clara Eugenia, Isabel de Borbón, Cristina de Suecia o Mariana de Austria. Disponible en YouTube https://t.co/ZBCcdsR7ub #ElPradoEnFemenino https://t.co/rHfVh59C3T
## -----
## 5. [2025-10-05] La serie documental "El Prado en femenino" cuenta con episodios dedicados a Isabel la Católica, María de Hungría, Isabel de Valois, Isabel Clara Eugenia, Isabel de Borbón, Cristina de Suecia o Mariana de Austria. Disponible en YouTube https://t.co/p490QbSs5W https://t.co/CCDJMQp2Tm
## -----
```

Red Local de Coocurrencias: 'femenino'



2.1.6 Word embeddings

- Modelo Word2Vec. El texto debe estar compuesto únicamente por los lemas deseados.
- Mostrar las similitudes de los términos: `"farnesio"`, `"caravaggio"`, `"università"`, `"falomir"`.
- Visualizar word embeddings mediante el método UMAP.

Empezamos preparando una lista de palabras limpia,

Hide

```
import pandas as pd
from gensim.models import Word2Vec
from umap import UMAP
import matplotlib.pyplot as plt

# 1. PREPARACIÓN DE DATOS (LEMAS)
docs_lemas = []
# Usamos prado_filtrado y la lista de stopwords que validaste antes
for doc in nlp.pipe(prado_filtrado["textmining"], disable=["ner"]):
    lemas = []
    for token in doc:
        if token.pos_ in ["NOUN", "ADJ", "PROPN"]:
            if token.lemma_.lower() not in stopwords:
                lemas.append(token.lemma_.lower())
    # Solo añadimos documentos que tengan al menos 2 palabras (para tener contexto)
    if len(lemas) > 1:
        docs_lemas.append(lemas)
```

Ahora realizamos el entrenamiento de la red

Hide

```
modelo_w2v = Word2Vec(
    docs_lemas,
    sg=1,                # Skip-gram: Ideal para corpus pequeños y relaciones raras
    vector_size=20,       # 20 dimensiones son suficientes para 800 tuits
    epochs=100,           # Subimos epochs para que converja mejor al ser pocos datos
    min_count=7,          # Aseguramos que entren las palabras del examen
    window=5,
    seed=42,
    workers=1             # workers=1 y seed fija aseguran reproducibilidad exacta
)
```

Hide

```
terminos_examen = ["farnesio", "caravaggio", "università", "falomir"]

print("\n3. SIMILITUDES VECTORIALES (Top 5 vecinos):")
```

```
##
## 3. SIMILITUDES VECTORIALES (Top 5 vecinos):
```

Hide

```
print("-" * 50)
```

```
## -----
```

Hide

```
for term in terminos_examen:
    try:
        similares = modelo_w2v.wv.most_similar(term, topn=5)
        print(f"► Vecinos de '{term.upper()}':")
        for palabra, similitud in similares:
            print(f"    - {palabra}: {similitud:.4f}")
    except KeyError:
        print(f"► [ALERTA] '{term.upper()}' no alcanzó el umbral mínimo de apariciones en el corpus.")
print()
```

```
## ► Vecinos de 'FARNESIO':  
##   - isabel: 0.9386  
##   - femenino: 0.8957  
##   - reina: 0.8808  
##   - 1692-1766: 0.8194  
##   - noelia: 0.7673  
##  
## ► Vecinos de 'CARAVAGGIO':  
##   - fresco: 0.7917  
##   - herrera: 0.7690  
##   - altura: 0.7340  
##   - resolución: 0.7160  
##   - panorámicas: 0.7064  
##  
## ► Vecinos de 'UNIVERSITÀ':  
##   - degli: 0.9649  
##   - pozzolo: 0.9644  
##   - studi: 0.9614  
##   - dal: 0.9590  
##   - maria: 0.9384  
##  
## ► Vecinos de 'FALOMIR':  
##   - director: 0.9269  
##   - miguel: 0.9260  
##   - enrico: 0.9094  
##   - maria: 0.8816  
##   - dal: 0.8402
```

Hide

```
vectores = modelo_w2v.wv.vectors  
vocabulario = modelo_w2v.wv.index_to_key  
  
# Calibración física de UMAP para separar bien las constelaciones  
reductor_umap = UMAP(n_neighbors=5, min_dist=0.5, metric='cosine', random_state=42)  
coordenadas_2d = reductor_umap.fit_transform(vectores)
```

Hide


```

limite_visual = min(60, len(vocabulario))
palabras_a_pintar = vocabulario[:limite_visual]

for term in terminos_examen:
    if term in vocabulario and term not in palabras_a_pintar:
        palabras_a_pintar.append(term)

indices = [vocabulario.index(palabra) for palabra in palabras_a_pintar]
x_coords = coordenadas_2d[indices, 0]
y_coords = coordenadas_2d[indices, 1]

# 1. Ajuste del lienzo (más ancho y con márgenes limpios como el profesor)
plt.figure(figsize=(14, 10))

# 2. Iteramos palabra por palabra para controlar exactamente cómo se dibuja cada una
for i, palabra in enumerate(palabras_a_pintar):
    if palabra in terminos_examen:
        # A) ESTÉTICA PALABRAS CLAVE (Examen)
        # Punto rojo más sutil
        plt.scatter(x_coords[i], y_coords[i], color='red', s=20, zorder=2, alpha=0.4)
        # Texto desplazado ligerísimamente hacia arriba, en negrita
        plt.text(x_coords[i], y_coords[i] + 0.08, palabra,
                 color='darkred', fontsize=6, fontweight='bold',
                 ha='center', va='bottom', zorder=6)
    else:
        # B) ESTÉTICA CONTEXTO (Ruido de fondo estilo profesor)
        # Círculo gris muy tenue, casi marca de agua
        plt.scatter(x_coords[i], y_coords[i], color="lightgray", s=20, edgecolor="darkgray", alpha=0.4, zorder=2)
        # Texto gris oscuro, fuente normal y más pequeña
        plt.text(x_coords[i], y_coords[i] + 0.05, palabra,
                 color='#4a4a4a', fontsize=6, fontweight='normal',
                 alpha=0.8, ha='center', va='bottom', zorder=3)

# 3. Título y Limpieza del gráfico (igual que Las coocurrencias del manual)
plt.title("Proyección UMAP de Word Embeddings",
          fontsize=12, fontweight="bold", loc="center")

# Ajuste fino de los márgenes para que respire
plt.subplots_adjust(left=0.05, right=0.95, top=0.90, bottom=0.05)
plt.axis("off") # Ocultamos los ejes X e Y, que en UMAP no tienen significado físico real

```

```

## (np.float64(-7.11599600315094), np.float64(5.346937298774719), np.float64(-5.974291753768921), np.float64(7.687
377882003784))

```

Hide

```
plt.show()
```



El modelo sitúa a *‘farnesio’* en un clúster muy cohesionado junto a *‘isabel’*, *‘femenino’*, *‘reina’* e *‘itinerario’*. Esto demuestra matemáticamente que la red neuronal ha “comprendido” el concepto de la exposición “El Prado en Femenino”, ligando el patronazgo de la reina con las visitas divulgativas.

Falomir y Università (Investigación/Veronese): Ambos términos gravitan en la zona superior de forma aislada, pero próximos a las fechas *‘1528-1588’* (Veronese). Esto corrobora el vínculo institucional del director del museo y la universidad italiana en el comisariado de dicha exposición.

Caravaggio (Aislamiento contextual): La palabra *‘caravaggio’* aparece en la periferia, sin un clúster fuerte a su alrededor. Las similitudes del coseno revelan que sus vecinos son *‘fresco’* o *‘altura’*, lo que sugiere menciones descriptivas puntuales y no una campaña temática estructurada.

3 Pregunta 2. Clasificación de Texto: Análisis de sentimiento

El dataset `reviews` contiene 13.000 críticas sobre el alojamiento en diversas ciudades de Andalucía.

3.1 Descripción de las columnas

- **título:** Breve descripción de la crítica.
- **texto:** El texto de la crítica.
- **puntuacion:** La calificación del usuario, en una escala de 5 estrellas. Variable categórica, con cinco valores posibles: 1 (muy mal), 2 (mal), 3 (regular), 4 (bien), 5 (muy bien).

3.2 Se pide

- Divida `reviews` en dos datasets: dataset de entrenamiento (con el 80% de las filas) y dataset de prueba (con el 20% de las filas restantes). No es necesario que los datasets estén balanceados.
- Mediante la herramienta `fastText`, cree un modelo de clasificación de texto basándose en la variable `puntuacion`.
- En `reviews`, cree una variable categórica, denominada `sentimiento`, siguiendo estas indicaciones:
 - Si `puntuacion` es menor de 3, tomará el valor negativo.
 - Si `puntuacion` es igual a 3, tomará el valor neutro.
 - Si `puntuacion` es mayor de 3, tomará el valor positivo.

- Empleando la herramienta `fastText`, cree un modelo de clasificación de texto basándose en la variable `sentimiento`.
- Utilizando la herramienta `fastText`, cree un modelo de clasificación de texto basándose en la variable `sentimiento`, pero seleccionando únicamente los valores negativo o positivo, es decir, prescindiendo de aquellas filas donde el sentimiento sea neutro.
- Empleando la Regresión Logística, cree un modelo de clasificación de texto basándose en la variable `sentimiento`, pero seleccionando únicamente los valores negativo o positivo.
- Usando el algoritmo Naive Bayes, cree un modelo de clasificación de texto basándose en la variable `sentimiento`, pero seleccionando únicamente los valores negativo o positivo.

4 Pregunta 3. Grandes Modelos de Lenguaje (LLM)

El dataset `documentos` contiene 120 oraciones.

4.1 Descripción de las columnas

- **doc_id:** Identificador del documento.
- **texto:** El texto del documento.

Sabemos de antemano que cada uno de los documentos pertenece a una de estas ocho categorías: Cine, Cocina, Deportes, Economía, Historia, Literatura, Naturaleza, Tecnología. De cada categoría hay 15 documentos.

4.2 Se pide

4.2.1 Clustering con BERT

Empleando Sentence Transformers, utilizar el modelo `hiiasid/sentence_similarity_spanish_es` para obtener los embeddings de los documentos.

- Visualizar los embeddings mediante UMAP.
- Realizar el clustering, asignando a cada documento una de las ocho categorías anteriormente indicadas.

4.2.2 Clustering con Nomic Bert

Empleando Ollama, usar el modelo `nomic-embed-text-v2-moe` para obtener los embeddings de los documentos.

- Visualizar los embeddings mediante UMAP.
- Realizar el clustering, asignando a cada documento una de las ocho categorías anteriormente señaladas.

4.2.3 Clasificación de texto con Qwen3

Empleando Ollama, usar el modelo `qwen3:4b` para clasificar los documentos, asignando a cada documento una de las ocho categorías anteriormente mencionadas.

Recomendaciones a seguir: No utilizar la salida estructurada, incluir en el prompt las ocho categorías. El thinking debe estar activado.

4.2.4 Comparar la clasificación de Qwen3 con el clustering de BERT

Mostrar aquellos documentos no coincidentes, es decir, la categoría asignada con Qwen3 es distinta al grupo asignado con BERT.

4.2.5 Comparar la clasificación de Qwen3 con el clustering de Nomic Bert

Mostrar aquellos documentos no coincidentes, es decir, la categoría asignada con Qwen3 es distinta al grupo asignado con Nomic Bert.

5 Pregunta 4. Grandes Modelos de Lenguaje (LLM)

Queremos extraer cierta información de varias páginas web de Wikipedia (URL), referentes a 10 personajes ilustres: "Albert Camus", "Marie Curie", "Albert Einstein", "Alexander Fleming", "Mahatma Gandhi", "Gabriel García Márquez", "Nelson Mandela", "Severo Ochoa", "Max Planck" y "Santiago Ramón y Cajal".

5.1 Páginas web (URL) a procesar

```
urls = [  
    "https://es.wikipedia.org/wiki/Albert_Camus",  
    "https://es.wikipedia.org/wiki/Marie_Curie",  
    "https://es.wikipedia.org/wiki/Albert_Einstein",  
    "https://es.wikipedia.org/wiki/Alexander_Fleming",  
    "https://es.wikipedia.org/wiki/Mahatma_Gandhi",  
    "https://es.wikipedia.org/wiki/Gabriel_García_Márquez",  
    "https://es.wikipedia.org/wiki/Nelson_Mandela",  
    "https://es.wikipedia.org/wiki/Severo_Ochoa",  
    "https://es.wikipedia.org/wiki/Max_Planck",  
    "https://es.wikipedia.org/wiki/Santiago_Ramón_y_Cajal"  
]
```

De cada personaje, queremos obtener la siguiente información:

- **Nombre:** Nombre de la persona.
- **Nacimiento:** Fecha y lugar de nacimiento.
- **Resumen:** Un resumen de su principal logro.
- **Nobel:** Disciplina en la que ganó el Premio Nobel.
- **Año:** Año en que ganó el Premio Nobel.
- **Fallecimiento:** Fecha y lugar de fallecimiento.

Se puede dar el caso, en el que algún personaje tenga más de un premio nobel (Marie Curie).

También, se puede dar el caso, en el que algún personaje nunca fue galardonado con el premio nobel (Mahatma Gandhi).

5.2 Se pide

- Utilizar la API Web fetch de Ollama para extraer el contenido de las páginas web indicadas.

5.2.1 Gemini

Utilizando la librería `chatlas` :

- Definir la salida estructurada a utilizar en el modelo.
- Usar el modelo Gemini para obtener la información requerida.

Es aconsejable hacer una pausa (30 o 45 segundos) entre peticiones, para no saturar el servidor.

5.2.2 Ollama

- Definir la salida estructurada a utilizar en el modelo.
- Usar el modelo `qwen3:4b` para obtener la información requerida.